



TED UNIVERSITY

**Faculty of Engineering
Department of Computer Engineering**

**CMPE 232 DATABASE SYSTEMS
Fall 2025**

TERM PROJECT DESIGN REPORT FOR RELATIONAL MODEL AND DATABASE STRUCTURES

Group Name	Section 2 - Group 2
Project Name/Subject	RecipeMatch+
Group Members (Name, ID, Dept)	Dilara Durgun 10001321 Computer Engineering
	Hacer Tunç 17897897972 Computer Engineering
	Deniz Kaya 10105201752 Computer Engineering
	Mariya Selin Yazıcı 10078712972 Software Engineering

I. Introduction

RecipeMatch+ is a relational database-driven program that suggests recipes based on what users already have in their pantry. Its main goals are to reduce food waste, save time, and provide a structured platform for users to create, search, rate, and bookmark recipes. A single, unique entity is represented by each table in our fully normalized schema, including User, Recipe, Ingredient, Rating, Bookmark, Pantry, and the weak object RecipeStep. For flexibility and consistency, common many-to-many interactions are resolved using bridge tables such as RecipeIngredient, Rating, Bookmark, and Pantry. IngredientCategory is a parent-child hierarchy to organize ingredients into important groups. In order to reduce complexity and prevent update problems, the database follows to Third Normal Form (3NF). Triggers maintain derived attributes, such as the average rating of a recipe, providing precise and reliable data. Data integrity is enforced and changes are tracked by unique keys, foreign keys, check constraints, and update timestamps. All things looked at, RecipeMatch+ offers a realistic and scalable relational model that facilitates efficient user interaction and recipe operation while adhering to practical database design standards.

II. Relational Model

a. Definition:

The relational model uses clear tables to describe users, recipes, ingredients, and their relationships. Foreign keys are used to create relationships in order to ensure data integrity. Because of normalization, each table contains an individual object and repetition is prevented.

The design consists of:

- 9 entities, all with auto-incremented IDs and timestamps.
- 6 relationships, including numerous many-to-many structures.
- A weak entity (RecipeStep) and a concept hierarchy (IngredientCategory).

b. Database Schema:

c.

A summary of the database tables and their purposes is provided below:

- **USER:** Stores account information for each user. Ensures unique identification through user_id and manages user interactions with recipes.
- **RECIPE:** Represents each recipe created in the system. Contains essential details such as cooking time, taste type, difficulty, and links to its creator via a foreign key.
- **INGREDIENT_CATEGORY:** Provides a hierarchical categorization system for ingredients. Supports parent-child relationships to structure ingredients into main and subcategories.
- **INGREDIENT:** Defines individual ingredients used in recipes. Includes metadata such as name and timestamps.
- **PANTRY:** Represents the ingredients owned by each user. Has a composite primary key (user_id + pantry_no) to store personal inventory data.

- **RECIPE_STEP (Weak Entity):** Stores step by step preparation instructions for each recipe. Depends on the Recipe table through a composite key (recipe_id + step_number).
- **STORED_IN:** Keeps the ingredients owned by the user in their own pantry. Represented by one to many relation. Has a composite primary key (user_id + pantry_no + ingredient_id).
- **RECIPE_INGREDIENT (CONTAINS):** A bridge table implementing the many to many relationships between recipes and ingredients. Stores ingredient amounts per recipe. Has a composite primary key (recipe_id + ingredient_id).
- **BOOKMARK:** Stores recipes saved by users. Represents the one to many interaction between users and the recipes they mark. Has a composite primary key (user_id + recipe_id).
- **RATING:** Stores user ratings for recipes. Links each rating to both a user and a recipe. Has a composite primary key (user_id + recipe_id).

d. Relationships:

A few key relationships in the system include:

- **One to Many Relationship (User to Recipe)**

One user can create many recipes, but each recipe is created by exactly one user. This relationship is established using a foreign key recipe.added_by referencing user.user_id.

- **One to Many Relationship (Recipe to RecipeStep)**

Each recipe must have one or more steps, while every step belongs to exactly one recipe. This relationship is established using a foreign key recipe_step.recipe_id referencing recipe.recipe_id.

- **Many to Many Relationship (Recipe to Ingredient)**

A recipe may contain many ingredients, and each ingredient can appear in multiple recipes. This relationship is established using the foreign keys recipe_ingredient.recipe_id referencing recipe.recipe_id and recipe_ingredient.ingredient_id referencing ingredient.ingredient_id.

- **Many to One Relationship (Ingredient to IngredientCategory)**

Each ingredient belongs to exactly one category, while a category may include many ingredients. This relationship is established using a foreign key ingredient.category_id referencing ingredient_category.category_id.

- **Zero to One to Zero to Zero to Many Relationship (IngredientCategory to SubCategoryOf)**

Recursive relationship representing main and subcategories. Each category can optionally have one parent and multiple children. Each category may have zero or one parent, and a parent category may have zero or many subcategories. This relationship is established using a self-referencing foreign key `ingredient_category.parent_id` referencing `ingredient_category.category_id`.

- **Many to Many Relationship (User to Pantry to Ingredient)**

Each user can have just one pantry. Users can store many ingredients, and the same ingredient can belong to multiple users' pantries. This relationship is established using the foreign keys `pantry.user_id` referencing `user.user_id` and `pantry.ingredient_id` referencing `ingredient.ingredient_id`.

- **Many to Many Relationship (User to Rating to Recipe)**

Users can rate many recipes, and recipes can receive many ratings. This relationship is established using the foreign keys `rating.user_id` referencing `user.user_id` and `rating.recipe_id` referencing `recipe.recipe_id`.

- **Many to Many Relationship (User to Bookmark to Recipe)**

Users can bookmark multiple recipes; recipes may be bookmarked by multiple users. This relationship is established using the foreign keys `bookmark.user_id` referencing `user.user_id` and `bookmark.recipe_id` referencing `recipe.recipe_id`.

e. Normalization:

- **USER: 3NF**
All non-key attributes depend solely on `user_id`; there are no partial or transitive dependencies.
- **RECIPE: 3NF**
Attributes such as `recipe_name`, `cook_time`, `difficulty` and `taste_type` depend only on `recipe_id`.
- **INGREDIENT: 3NF**
Each ingredient has atomic attributes, and `category_id` is a valid foreign key that does not introduce redundancy.
- **INGREDIENT_CATEGORY: 3NF**
The self-referencing `parent_id` attribute preserves hierarchy without violating normalization rules.
- **PANTRY: 2NF**
The composite primary key (`user_id`, `ingredient_id`) fully determines quantity and unit; there are no repeating groups.
- **RECIPE_STEP (Weak Entity): 2NF**
The composite key (`recipe_id`, `step_number`) removes partial key dependencies, and each step attribute depends on the whole key.
- **RECIPE_INGREDIENT (Contains): 2NF**
Stores only atomic values such as amount and unit; all depend on the composite key (`recipe_id`, `ingredient_id`).
- **STORED_IN: 3NF**

Represents the storage relationship between user, pantry and ingredient. The composite key uniquely identifies each record, and all attributes depend on that key, with no transitive dependencies.

- **RATING: 2NF**
The combination of user_id and recipe_id determines the rating_value; there are no additional derived or repeating attributes.
- **BOOKMARK: 2NF**
The primary key (or composite key of user_id and recipe_id) is the only determinant; all attributes depend directly on it.

This normalization ensures data integrity, minimal redundancy, and improves query performance.

III. Database Structures

A simple and carefully written summary of every database structure utilized in RecipeMatch+ is given in this section. Each part is thoroughly described to show its function, technical accuracy, and contribution to the overall integrity of the system.

Table Structures:

All the tables used in the RecipeMatch+ project are listed below, along with their columns, data types, constraints, and important relationships.

1. User

Allows for personalization and maintains basic data about every signed user.

- **Columns & Data Types**
 - user_id INT – auto-increment
 - username VARCHAR(50)
 - email VARCHAR(100)
 - password VARCHAR(100)
 - created_at DATETIME
 - modified_at DATETIME
- **Constraints**
 - user_id is **PRIMARY KEY**, auto-increment and **NOT NULL**.
 - username is **UNIQUE** and **NOT NULL**.
 - email is **UNIQUE** and **NOT NULL**.
 - password is **NOT NULL**.
 - created_at and modified_at are **NOT NULL**
- **Relationships**
 - Referenced by Recipe.user_id.
 - Referenced by Rating.user_id, Bookmark.user_id, and Pantry.user_id.

2. Recipe

Represents a user-created recipe together with its essential properties and metadata.

- **Columns & Data Types**

- recipe_id INT – auto-increment
- recipe_name VARCHAR(100)
- cook_time INT
- difficulty VARCHAR(10)
- taste_type VARCHAR(10)
- added_by INT -- FK to User
- created_at DATETIME
- modified_at DATETIME
- **Constraints**
 - recipe_id is **PRIMARY KEY**, auto-increment.
 - recipe_name is **NOT NULL**.
 - cook_time is **NOT NULL** and typically positive (CHECK (cook_time > 0)).
 - difficulty is **NOT NULL** with CHECK (difficulty IN ('easy', 'medium', 'hard')).
 - taste_type is **NOT NULL**, with CHECK (taste_type IN ('sweet', 'salty', 'spicy', 'sour')).
- **Relationships**
 - user_id is a **FOREIGN KEY** referencing User(user_id) (ensures each recipe belongs to a registered user).
 - Referenced by Rating.recipe_id, Bookmark.recipe_id, RecipeIngredient.recipe_id, and RecipeStep.recipe_id.

3. Ingredient

Represents a single ingredient that can appear in recipes or user pantries.

- **Columns & Data Types**
 - ingredient_id INT – auto-increment
 - ingredient_name VARCHAR(100)
 - category_id INT -- FK to IngredientCategory (optional)
 - created_at DATETIME
 - modified_at DATETIME
- **Constraints**
 - ingredient_id is **PRIMARY KEY**, auto-increment.
 - ingredient_name is **NOT NULL**.
 - category_id is **NULLABLE** (an ingredient may temporarily have no category).
- **Relationships**
 - category_id is a **FOREIGN KEY** referencing IngredientCategory(category_id).
 - Referenced by Pantry.ingredient_id and RecipeIngredient.ingredient_id.

4. IngredientCategory

Categorizes ingredients into major and subcategories using the idea hierarchy.

- **Columns & Data Types**
 - category_id INT – auto-increment
 - category_name VARCHAR(100)
 - parent_category_id INT -- self-FK
 - created_at DATETIME
 - modified_at DATETIME
- **Constraints**
 - category_id is **PRIMARY KEY**, auto-increment.
 - category_name is **NOT NULL**.

- parent_category_id is **NULLABLE** (top-level categories have no parent).
- **Relationships**
 - parent_category_id is a **FOREIGN KEY** referencing IngredientCategory(category_id) (self-referencing hierarchy).
 - Referenced by Ingredient.category_id.

5. Rating

Maintains feedback from users for recipes on a scale of 1 to 5 stars, with the business standard being one rating per user every recipe.

- **Columns & Data Types**
 - user_id INT
 - recipe_id INT
 - stars INT -- {1,2,3,4,5}
 - created_at DATETIME
 - modified_at DATETIME
- **Constraints**
 - Composite **PRIMARY KEY**: (user_id, recipe_id) to enforce “for every user just one rating per recipe”.
 - stars is **NOT NULL** with CHECK (stars BETWEEN 1 AND 5).
- **Relationships**
 - user_id is a **FOREIGN KEY** referencing User(user_id).
 - recipe_id is a **FOREIGN KEY** referencing Recipe(recipe_id).

6. Bookmark

Stores recipes that users mark as favorites.

- **Columns & Data Types**
 - user_id INT
 - recipe_id INT
 - created_at DATETIME
 - modified_at DATETIME
- **Constraints**
 - Composite **PRIMARY KEY**: (user_id, recipe_id) – prevents a user from bookmarking the same recipe more than once.
- **Relationships**
 - user_id is a **FOREIGN KEY** referencing User(user_id).
 - recipe_id is a **FOREIGN KEY** referencing Recipe(recipe_id).

7. Pantry

Controls a suggestion logic and lists the ingredients that each user owns.

- **Columns & Data Types**
 - user_id INT
 - pantry_no INT

- created_at DATETIME
 - modified_at DATETIME
- **Constraints**
 - **PRIMARY KEY:** (user_id, pantry_no).
- **Relationships**
 - user_id is a **FOREIGN KEY** referencing User(user_id).

8. RecipeIngredient

Maintains the amount of each ingredient needed and links recipes and ingredients.

- **Columns & Data Types**
 - recipe_id INT
 - ingredient_id INT
 - amount_grams DECIMAL(10,2)
- **Constraints**
 - Composite **PRIMARY KEY:** (recipe_id, ingredient_id).
 - Amount_grams is **NOT NULL** to ensure well-defined recipe composition.
- **Relationships**
 - recipe_id is a **FOREIGN KEY** referencing Recipe(recipe_id).
 - ingredient_id is a **FOREIGN KEY** referencing Ingredient(ingredient_id).

9. RecipeStep (Weak Entity)

The order recipe production rules are stored by a weak entity. Without a parent recipe, it would not be possible.

- **Columns & Data Types**
 - recipe_id INT
 - step_number INT
 - step_text TEXT
 - created_at DATETIME
 - modified_at DATETIME
- **Constraints**
 - Composite **PRIMARY KEY:** (recipe_id, step_number).
 - step_text is **NOT NULL**.
- **Relationships**
 - recipe_id is a **FOREIGN KEY** referencing Recipe(recipe_id) and makes RecipeStep a weak entity dependent on Recipe.

10. StoredIn

The order recipe production rules are stored by a weak entity. Without a parent recipe, it would not be possible.

- **Columns & Data Types**
 - user_id INT
 - pantry_no INT
 - ingredient_id INT
 - quantity DECIMAL(10,2)
- **Constraints**
 - Composite **PRIMARY KEY:** (user_id, pantry_no, ingredient_id).
 - User_id, pantry_no and ingredient_id are **NOT NULL**.

- **Relationships**

- user_id and pantry_no are **FOREIGN KEYS** referencing Pantry(user_id, pantry_no).
- ingredient_id is a **FOREIGN KEY** referencing Ingredient(ingredient_id).

Indices:

- Identify any indices created on tables to improve query performance.
- Primary Key Indices

Primary keys automatically generate unique indices, allowing fast row lookup and efficient access paths.

- USER (user_id)
- RECIPE(recipe_id)
- INGREDIENT_CATEGORY(category_id)
- INGREDIENT(ingredient_id)
- PANTRY (user_id, pantry_no) – composite index
- RECIPE_STEP (recipe_id, step_number) – composite index
- RECIPE_INGREDIENT (recipe_id, ingredient_id) – composite index
- BOOKMARK (user_id, recipe_id) – composite index
- RATING (user_id, recipe_id) – composite index

- **Unique Indices**

It contains automatically generated indices that prevent duplicate values and improve search performance.

- USER (username)
- USER(email)
- INGREDIENT(ingredient_name)
- INGREDIENT_CATEGORY(category_name)

- **Foreign Key Indices**

Each foreign key column is indexed to provide efficient relational lookups and fast access paths between interconnected entities.

- RECIPE(user_id)
- INGREDIENT(category_id)
- PANTRY(user_id)
- STORED_IN(user_id, pantry_no) – composite foreign key index
- STORED_IN(ingredient_id)
- RECIPE_INGREDIENT(recipe_id)
- RECIPE_INGREDIENT(ingredient_id)
- BOOKMARK(user_id)
- BOOKMARK(recipe_id)
- RATING(user_id)
- RATING(recipe_id)

Views:

In our project, a recipe's average rating (avg_rating) is a derived attribute that is not held in the RECIPE table. Holding it would create unnecessary duplication and inconsistencies. The

average rating is calculated using a view, which aggregates the RATING table.

```
CREATE VIEW recipe_rating_summary AS
SELECT
    r.recipe_id,
    r.recipe_name,
    CAST(AVG(rt.stars) AS DECIMAL (10, 2)) AS avg_rating,
    COUNT(rt.users_id) AS total_values
FROM RECIPE r
LEFT JOIN RATING rt
    ON r.recipe_id = rt.recipe_id
GROUP BY r.recipe_id, r.recipe_name;

SELECT *
FROM recipe_rating_summary
```

Optionally, calculating the avg_rating removes redundancy and reflects the current state of the data. The average rating gets updated as new ratings are entered and as ratings are modified, which is why there is no need for a stored field or a trigger.

This view is for every ingredient in a recipe, we assign points depending on what the user will get 1 point if the user has the exact ingredient, 0.5 points if they have something from the same ingredient category and 0 points if they have nothing related. Then we convert the total points into a percentage. This gives us a “match score” like “78% match with this recipe.” It’s what we use to recommend recipes that fit the user’s available ingredients.

```
CREATE VIEW USER_RECIPE_MATCH AS
SELECT u.users_id, r.recipe_id, r.recipe_name,
CAST( ROUND ( ( SUM(CASE
    WHEN si.ingredient_id=r_ing.ingredient_id THEN 1
    WHEN cat_pantry.category_id=cat_recipe.category_id THEN 0.5
    ELSE 0
    END)/COUNT(r_ing.ingredient_id))*100,0) AS INT) AS match_score
FROM USERS u
CROSS JOIN RECIPE r
JOIN RECIPE_INGREDIENT r_ing ON r_ing.recipe_id=r.recipe_id
LEFT JOIN PANTRY p ON p.users_id=u.users_id
LEFT JOIN STORED_IN si ON si.users_id=p.users_id AND
si.pantry_no=p.pantry_no
LEFT JOIN INGREDIENT ing_pantry ON
si.ingredient_id=ing_pantry.ingredient_id
LEFT JOIN INGREDIENT_CATEGORY cat_pantry ON
ing_pantry.category_id=cat_pantry.category_id
LEFT JOIN INGREDIENT ing_recipe ON
r_ing.ingredient_id=ing_recipe.ingredient_id
LEFT JOIN INGREDIENT_CATEGORY cat_recipe ON
ing_recipe.category_id=cat_recipe.category_id
GROUP BY u.users_id, r.recipe_id, r.recipe_name;

SELECT *
FROM USER_RECIPE_MATCH
```

```
ORDER BY users_id ASC, match_score DESC;
```

Triggers:

Triggers played an important role in automating the management of timestamps and recording the updates of recipes correctly. In this project, the triggers were used for updating the `modified_at` field and thus enabling the system to record both direct edits to a recipe as well as indirect changes caused by user activity. Also to make sure we never store negative quantities in the pantry.

Three triggers were used in this project.

The first trigger is set to automatically update the recipe when its contents are modified. This trigger fires before any `UPDATE` stored in the `RECIPE` table, changing the `modified_at` value of the record being modified to the current timestamp. This ensures that the updated content of the recipe is created correctly without manual modification. Furthermore, if the user provided ratings are not changed, adding a new record to the `RATING` table will not trigger this trigger; thus, unnecessary timestamp updates are prevented.

```
CREATE TRIGGER update_recipe_modified_at ON RECIPE
AFTER UPDATE
AS
    UPDATE RECIPE
    SET modified_at = CURRENT_TIMESTAMP
    WHERE recipe_id IN (SELECT recipe_id FROM inserted);
```

A second trigger was created to capture indirect recipe updates caused by new user ratings. Although the average rating of a recipe is a derived attribute computed by a view, submitting a new rating still constitutes a meaningful user interaction. To reflect this interaction in the recipe's metadata, a trigger updates the recipe's `modified_at` field whenever a new rating is inserted into the `RATING` table.

```
CREATE TRIGGER update_recipe_modified_on_rating ON RATING
AFTER INSERT
AS
    UPDATE RECIPE
    SET modified_at = CURRENT_TIMESTAMP
    WHERE recipe_id IN (SELECT recipe_id FROM inserted);
```

The third trigger makes sure we never store negative quantities in the pantry. If someone accidentally tries to update a quantity to a negative value, the trigger automatically sets it to zero. It's a simple safeguard to keep the data clean and prevent weird cases like “-4 eggs” or “-20 grams of salt.”

```
CREATE TRIGGER prevent_negative_stock ON STORED_IN
AFTER UPDATE
AS
    UPDATE s
    SET s.quantity=0
    FROM STORED_IN s
    JOIN inserted i ON s.users_id=i.users_id AND
    s.pantry_no=i.pantry_no AND s.ingredient_id=i.ingredient_id
```

```
WHERE i.quantity<0;
```

By detailing each of these database structures along with relevant SQL statements, users can gain a clear understanding of the underlying architecture and functionality of the RDB project. Additionally, utilizing tools like Windows Forms, WPF, cshtml, etc. to visualize and manage these structures can enhance the development and maintenance process.

IV. UI Design

Login / Registration Page

This page serves as the main point of entry for users to register for a new account or log in. Depending on their function, it takes users to either the Home Page or the Admin Page and has two different panels. These role-based navigation methods are clearly shown in the diagram.

Home page

Users' main navigation screen is the Home Page. It provides easy access to Recommended Recipes, Bookmarked Recipes, and My Pantry in addition to recipe search and filtering capabilities. All of the navigation routes from the Home Page to other important screens are shown in the diagram.

Recipe List Page

A list of recipes is shown on this page according to the user's search or filters. When necessary, users can go back to the Home Page or select a recipe to view its specifics. The figure provides a clear representation of these transitions.

Recipe Detail Page

This page shows every detail of a given recipe, such as ingredients, preparation methods, level of difficulty, and cooking time. The recipe can be rated, bookmarked, or returned to the Recipe List. All the navigation points to and from this page are shown in the figure.

Pantry Management Page

Users can add, modify, or arrange the ingredients that are on offer using the Pantry page. It is possible to search, classify, and assign quantities to items. The links connecting this page, the Home Page, and other relevant screens are shown in the graphic.

Rating Page

Users can rate recipes and see the average score on the Rating Page. After rating a recipe, users can return to the recipe's details. The interface diagram makes this link obvious.

Bookmark Page

All the user's stored recipes are shown on this page. Users can see recipe details or delete things from their bookmark list from this location. The navigation routes between the Home Page, Recipe Detail Page, and Bookmark Page are shown in the diagram.

Manage Account Page

Users can modify their account details, including name, email address, and password, on this screen. Users can go back to the Home Page after verifying the modifications. The relationship and navigation method is shown in the diagram.

Admin Page

For managers, the Admin Page serves as their primary control center. SQL queries, preset query tools, ingredient management, recipe management, and managing users are all accessible. The figure displays every admin-related transition.

Manage Users Page

Users can view the user list and delete specific accounts from this site. The updated list is shown in the results area, and navigation goes back to the Admin Page. These relationships are clearly shown in the diagram.

Manage Recipes Page

Users can add, edit, or remove recipes from this page. modify recipe fields include ingredient lists, cook time, and difficulty. This page's connections to other admin tools are shown in the diagram.

Manage Ingredients Page

Administrators can add, modify, or remove ingredients using this page. The searchable list allows for the management and classification of ingredients. Its travel method back to the Admin Page is shown in the diagram.

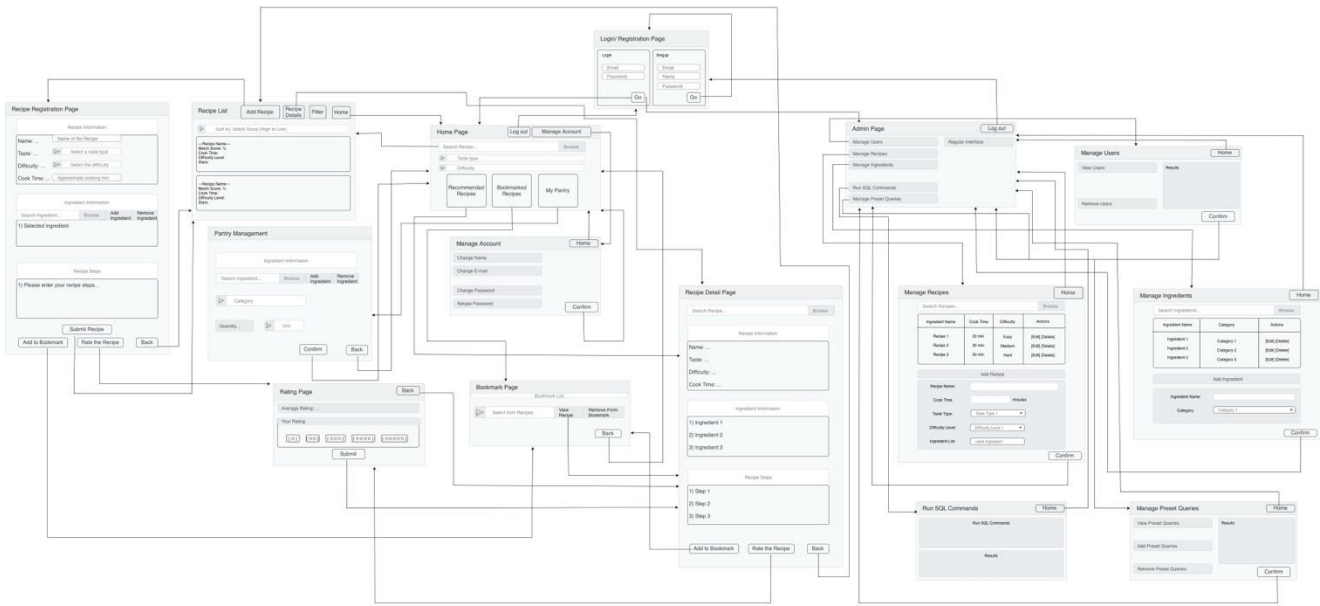
Manage Preset Queries Page

This site allows administrators to see, add, or remove preset SQL queries. The diagram displays connections to the Admin Page, and the results are shown under the question list.

Run SQL Commands Page

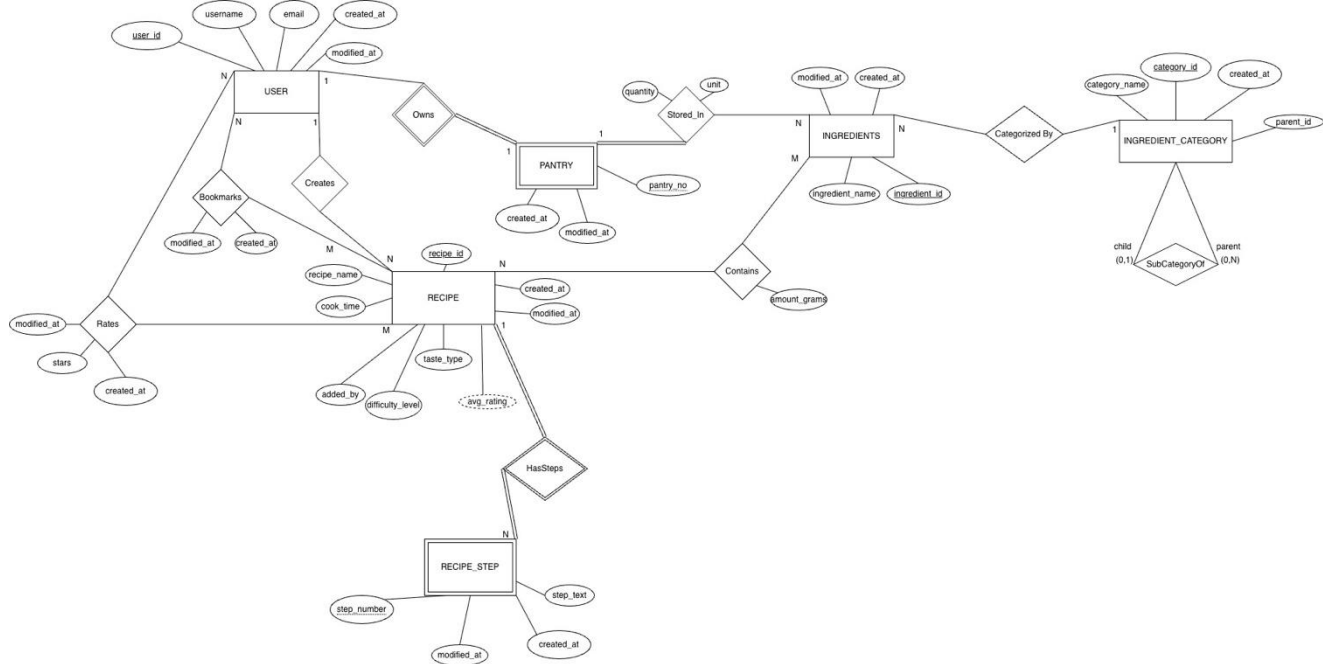
Users can manually enter and run SQL directions on this page. The output appears right away, and the Home button takes users back to the Admin Page. This method is clearly shown in the diagram.

UI Design Flow



V. E-R Diagram

Bellow, we updated our E-R diagram with the feedback we've received from our teachers.



VI. Workload Division

Task Name	Responsible Member(s)
- Database implementation - Presentation of the project	All members
RecipeIngredient, Rating, Bookmark	Mariya Selin Yazıcı
Ingredient, IngredientCategory	Hacer Tunç
User, Pantry, StoredIn	Dilara Durgun
Recipe, RecipeStep	Deniz Kaya

VII. References

A free database designer for developers and analysts. Database Relationship Diagrams Design Tool. (n.d.). <https://dbdiagram.io/>

Elmasri, R., & Navathe, S. B. (2016). *Fundamentals of Database Systems (7th ed.)*. Pearson.

Flowchart Maker and diagrams online. Flowchart Maker & Online Diagram Software. (n.d.). <https://www.draw.io/>